AULA • BANCO DE DADOS

UPDATE e DELETE no MySQL

Atualização e exclusão segura de dados relacionais

Onde são utilizados e por que são importantes

UPDATE e DELETE são comandos de manipulação de dados (DML). UPDATE modifica valores de registros existentes, DELETE remove registros.

Eles aparecem em sistemas de vendas, cadastros, estoque, financeiro, atendimento e em rotinas administrativas.

São importantes porque mantêm os dados corretos ao longo do tempo: corrigem cadastros, alteram status de pedidos, atualizam preços e removem registros que não devem mais permanecer. Como podem afetar muitos dados de uma só vez, exigem validação, controle de acesso e procedimentos de recuperação.

> **Regra de segurança:** antes de alterar ou excluir, identifique exatamente quais linhas serão atingidas. Em produção, siga o processo de mudança da organização e tenha um backup/restauração testados.

Base de exemplo

Os exemplos usam uma tabela de vendas simplificada. Em uma atividade prática, crie a tabela e insira os dados em um banco de testes, não em uma base de produção.

```
CREATE DATABASE aula_mysql,
USE aula_mysql,

CREATE TABLE vendas (
    id INT PRIMARY KEY AUTO_INCREMENT,
    vendedor VARCHAR(100) NOT NULL,
    produto VARCHAR(100) NOT NULL,
    categoria VARCHAR(50),
    quantidade INT NOT NULL,
    preco DECIMAL(10,2) NOT NULL,
    data_venda DATE NOT NULL,
    desconto DECIMAL(5,2) DEFAULT 0,
    observacao VARCHAR(255) NULL
) ENGINE=InnoDB,
```

```
INSERT INTO vendas
(vendedor, produto, categoria, quantidade, preco, data_venda, desconto, observacao)
VALUES
('Ana', 'Notebook', 'Informática', 2, 3500.00, '2026-01-10', 5.00, NULL),
('Bruno', 'Mouse', 'Informática', 10, 80.00, '2026-01-11', 0.00, 'Venda à vista'),
('Ana', 'Teclado', 'Informática', 5, 150.00, '2026-01-12', 10.00, NULL),
('Carlos', 'Cadeira', 'Móveis', 3, 900.00, '2026-01-13', 8.00, 'Entrega pendente'),
('Bruno', 'Mesa', 'Móveis', 2, 1200.00, '2026-01-14', 0.00, NULL),
('Carlos', 'Monitor', 'Informática', 4, 1100.00, '2026-01-15', 5.00, NULL),
```

UPDATE: alterando dados

A instrução UPDATE modifica uma ou mais colunas das linhas selecionadas. A forma geral é:

```
UPDATE nome_tabela
SET coluna1 = valor1, coluna2 = valor2
WHERE condição,
```

Exemplo: corrigir o preço do Mouse identificado pelo produto.

```
UPDATE vendas
SET preco = 85.00
WHERE produto = 'Mouse',
```

É preferível filtrar pela chave primária quando ela estiver disponível, pois o identificador aponta para uma linha específica:

```
UPDATE vendas
SET observacao = 'Conferir pagamento'
WHERE id = 2,
```

Mais de uma coluna pode ser atualizada na mesma operação:

```
UPDATE vendas
SET quantidade = 12, observacao = 'Quantidade conferida'
WHERE id = 2,
```

Atualizações com expressões e condições

O lado direito do SET pode usar o valor atual da própria linha. Por exemplo, aplicar aumento de 5% à categoria Móveis:

```
UPDATE vendas
SET preco = preco * 1.05
WHERE categoria = 'Móveis',
```

Atualizar somente vendas a partir de uma data, usando mais de uma condição:

```
UPDATE vendas
SET desconto = 7.00
WHERE categoria = 'Informática'
  AND data_venda >= '2026-01-12',
```

Também é possível atribuir valores diferentes conforme uma condição usando CASE:

```
UPDATE vendas
SET observacao = CASE
    WHEN quantidade >= 5 THEN 'Volume alto'
    ELSE 'Volume padrão'
END
WHERE categoria = 'Informática',
```

Atenção: o UPDATE sem WHERE atualiza todas as linhas da tabela. Só faça isso quando essa for a intenção explícita e após validar o impacto.

DELETE: removendo dados

DELETE remove linhas da tabela. A condição WHERE determina quais registros serão excluídos:

```
DELETE FROM vendas
WHERE id = 5,
```

É possível excluir um conjunto de linhas usando uma condição cuidadosamente definida:

```
DELETE FROM vendas
WHERE data_venda < '2026-01-12',
```

Se a condição não identificar nenhuma linha, nenhuma linha será removida. Se a condição for ampla demais, serão removidas mais linhas do que o esperado.

Perigo: DELETE FROM vendas, sem WHERE apaga todas as linhas da tabela. A estrutura da tabela permanece, mas os dados são removidos.

Validar antes de executar: SELECT e contagem

Uma prática simples e eficaz é converter o filtro do UPDATE ou DELETE em um SELECT antes de executar a alteração. Assim, é possível conferir as linhas-alvo.

```
SELECT id, vendedor, produto, preco
FROM vendas
WHERE categoria = 'Móveis',

SELECT COUNT(*) AS linhas_afetadas
FROM vendas
WHERE categoria = 'Móveis',
```

Depois de revisar o resultado, use exatamente a mesma condição no UPDATE ou DELETE. Em ferramentas como MySQL Workbench, confira também a mensagem de linhas afetadas, ela é uma verificação adicional, não substitui a validação do filtro.

Transações: COMMIT e ROLLBACK

Uma transação permite agrupar operações e confirmar ou desfazer as alterações. Nas tabelas InnoDB, as operações DML podem participar de transações. START TRANSACTION inicia a transação, COMMIT confirma, ROLLBACK desfaz alterações ainda não confirmadas.

```
START TRANSACTION,

UPDATE vendas
SET desconto = 6.00
WHERE id = 1,

SELECT id, produto, desconto
FROM vendas
```

```
WHERE id = 1,

-- Se o resultado estiver correto:
COMMIT,

-- Se encontrar um erro antes do COMMIT, use ROLLBACK:
-- ROLLBACK,
```

Exclusão revisável dentro de uma transação:

```
START TRANSACTION,

DELETE FROM vendas
WHERE id = 6,

-- Confira o estado resultante antes de confirmar:
SELECT * FROM vendas WHERE id = 6,

-- Confirmar somente depois da validação:
COMMIT,
-- Caso contrário, antes do COMMIT: ROLLBACK,
```

Limite importante: COMMIT torna a alteração persistente e ROLLBACK não desfaz uma transação já confirmada. DDL, como ALTER TABLE, pode provocar commit implícito no MySQL, não misture mudanças de estrutura com a expectativa de que tudo será revertido.

DELETE, TRUNCATE e DROP: não são a mesma coisa

DELETE remove linhas e pode usar WHERE.

TRUNCATE TABLE esvazia a tabela inteira sem filtro por linha.

DROP TABLE remove a própria tabela, incluindo sua estrutura.

TRUNCATE e DROP são operações de DDL e têm comportamento transacional/recuperação distinto do DELETE, não devem ser tratados como substitutos seguros de uma exclusão seletiva.

```
-- Remover linhas selecionadas:
DELETE FROM vendas WHERE id = 3,

-- Esvaziar a tabela inteira (operação ampla):
TRUNCATE TABLE vendas,

-- Remover a tabela e sua estrutura:
DROP TABLE vendas,
```

Para a rotina normal de exclusão de registros, use DELETE com um critério claro. Use TRUNCATE ou DROP apenas quando a operação completa for deliberada e autorizada.

Chaves estrangeiras e efeitos relacionados

Em bancos relacionais, outras tabelas podem referenciar a linha que se deseja excluir. Uma chave estrangeira pode impedir a exclusão ou propagar a remoção quando foi configurada com ON DELETE CASCADE. O comportamento depende do esquema, verifique as relações antes de apagar dados.

A cascata pode ser útil para remover dependências que não devem existir sem o registro pai, mas também pode apagar muitos registros relacionados. Confirme a regra com o responsável pelo banco e faça a revisão do impacto.

Erros comuns e como evitá-los

Esquecer o WHERE: Pode atualizar ou apagar todas as linhas. Revise o filtro e confira o alvo com SELECT.

Usar um filtro pouco específico: Condições como categoria = ... podem atingir muitas linhas. Verifique a contagem e, se apropriado, use a chave primária.

Não conferir o tipo de dado: Use valores compatíveis com a coluna, datas no formato ISO AAAA-MM-DD e decimais com ponto no SQL.

Esperar que ROLLBACK reverta COMMIT: Depois de confirmado, é necessário um procedimento de restauração ou uma correção compensatória.

Desativar restrições sem necessidade: Pode deixar referências inconsistentes. Não desabilite verificações de integridade como atalho.

Executar direto em produção: Teste em ambiente controlado, aplique privilégios mínimos e use o processo de backup/restauração vigente.

Exercício de exemplo

Use a tabela vendas criada nesta aula.

Antes de executar alterações ou exclusões, escreva um SELECT com o mesmo filtro e verifique as linhas.

1. Atualize o preço do Mouse para R$ 82,50 usando a chave primária correspondente.
2. Altere a observação da venda de id 4 para “Entrega confirmada”.
3. Aplique desconto de 3% às vendas da categoria Móveis.
4. Atualize observacao para “Revisar venda” em vendas com quantidade igual ou superior a 5.
5. Dentro de uma transação, altere o preço do Notebook para R$ 3.600,00, confira o valor e desfaça a alteração com ROLLBACK.
6. Exclua somente a venda de id 5, após verificar com SELECT qual produto será afetado.
7. Explique o que acontece se executar UPDATE vendas SET desconto = 0,
8. Explique a diferença entre DELETE FROM vendas, e TRUNCATE TABLE vendas.

9. Escreva um comando para excluir vendas anteriores a 2026-01-12. Antes, mostre o SELECT de validação.
10. Em uma tabela de pedidos e itens_pedido com chave estrangeira, explique o que deve ser verificado antes de excluir um pedido.

13. Gabarito sugerido

```
-- 1
UPDATE vendas SET preco = 82.50 WHERE id = 2,

-- 2
UPDATE vendas SET observacao = 'Entrega confirmada' WHERE id = 4,

-- 3
UPDATE vendas SET desconto = 3.00 WHERE categoria = 'Móveis',

-- 4
UPDATE vendas SET observacao = 'Revisar venda' WHERE quantidade >= 5,

-- 5 (confira antes e depois, ROLLBACK desfaz)
START TRANSACTION,
UPDATE vendas SET preco = 3600.00 WHERE id = 1,
SELECT id, produto, preco FROM vendas WHERE id = 1,
ROLLBACK,

-- 6: validar e depois remover
SELECT id, produto FROM vendas WHERE id = 5,
DELETE FROM vendas WHERE id = 5,

-- 7: define desconto como zero para todas as linhas, pois não há WHERE.

-- 8: DELETE remove linhas e aceita WHERE, TRUNCATE esvazia a tabela inteira.

-- 9: validar primeiro
SELECT id, data_venda FROM vendas WHERE data_venda < '2026-01-12',
DELETE FROM vendas WHERE data_venda < '2026-01-12',
```

Roteiro mental antes de alterar ou excluir

- Estou conectado ao banco correto e ao ambiente esperado?
- Consigo identificar as linhas com um SELECT usando a condição exata?
- A quantidade de linhas encontradas corresponde ao esperado?
- Há chaves estrangeiras, cascatas ou dependências envolvidas?
- Tenho autorização, backup/restauração disponível e plano de reversão?
- Se usar transação, vou revisar antes do COMMIT?

15. Resumo

- UPDATE tabela SET coluna = valor WHERE condição, atualiza dados existentes.
- DELETE FROM tabela WHERE condição, remove linhas selecionadas.

- WHERE é essencial para limitar o impacto, sem ele, todos os registros podem ser atingidos.
- SELECT e COUNT(*) ajudam a revisar o alvo antes da operação.
- START TRANSACTION, COMMIT e ROLLBACK apoiam alterações controladas em tabelas InnoDB.
- DELETE não é igual a TRUNCATE ou DROP, cada comando tem alcance e efeitos diferentes.
- Em produção, combine validação, privilégios adequados, processo de mudança e recuperação testada.